

DATA-260 Section 21/71 — Homework 2
Haozhe Wang

SID4: 8561
PORT_BASE: 8461
PREFIX: s8561
SEED: 8561
VERIFY_SEED: 268561
DOMAIN_ID: 1

Hardware: Apple MacBook Pro, M4, 16 GB unified memory
Local model: qwen3:8b (served locally by Ollama)
Commit hash: 2cad9038ff124785d7bb21e2ebdac9ff3a6a9de5

Repository: <https://github.com/HauZer-HaoZheWang/data260-8561>
Collaborators Sbnikitha and supriyaselvangesan have access to this repository.

PART 1 — HTML & CSS

1.1 Responsive layout at 375px

Code — `static/styles.css`

```
/*Include padding and border inside element width*/
* {
  box-sizing: border-box;
}

/*Let only the table area scroll on a narrow screen*/
.tableWrapper {
  width: 100%;
  max-width: 100%;
  overflow-x: auto;
}

table {
  width: 100%;
  min-width: 620px;
  border-collapse: collapse;
}

/*Apply these styles only on narrow screens*/
@media (max-width: 480px) {
```

```
main { padding: 10px; }
h1 { font-size: 24px; }
h2 { font-size: 20px; }
section { padding: 12px; }

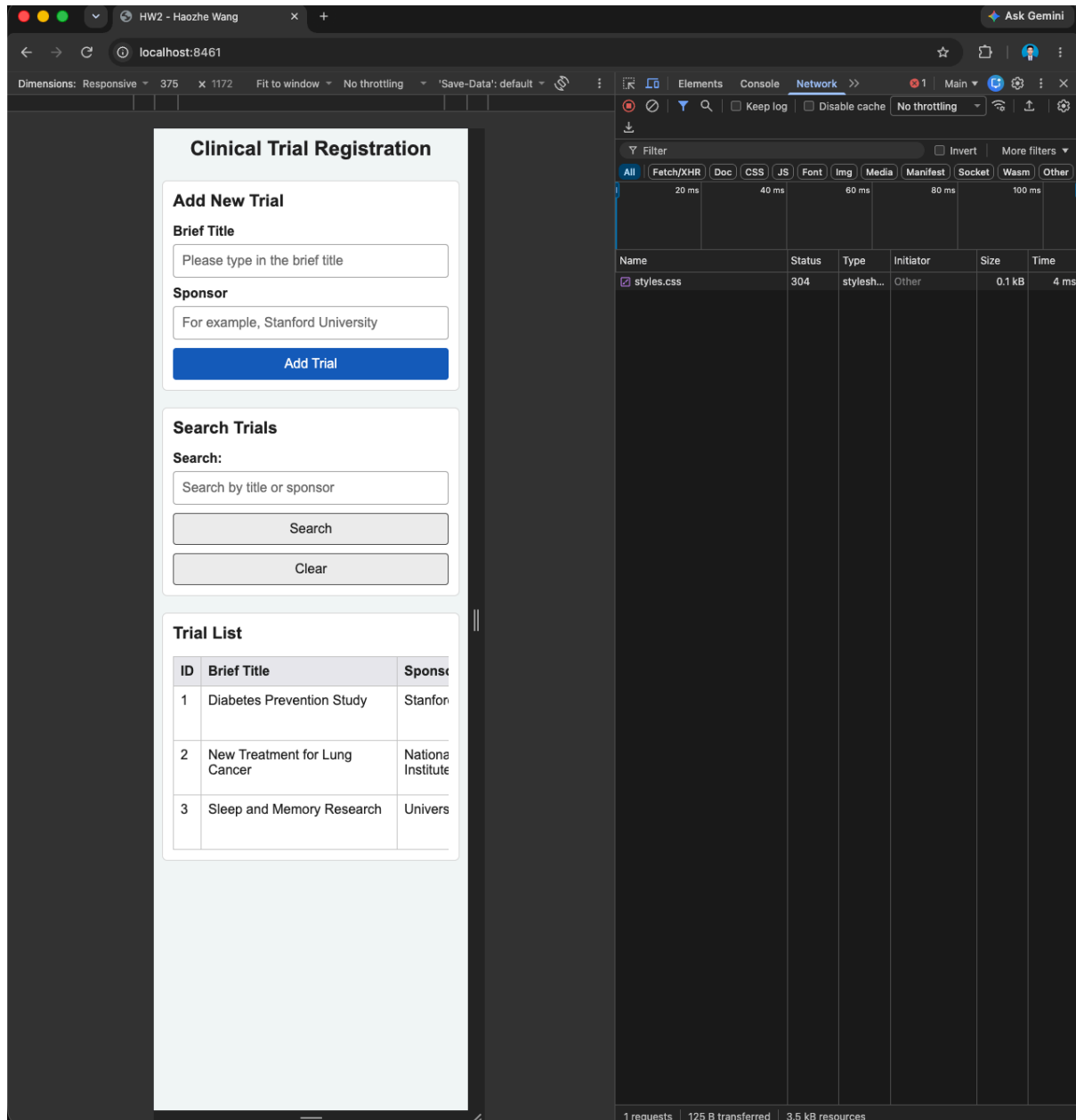
#createForm button,
#searchButton,
#clearSearchButton {
  width: 100%;
  margin-right: 0;
}

th, td { padding: 8px; }
}
```

Code — static/index.html

```
<div class="tableWrapper">
  <table id="trialTable" style="display: none;">
    ...
  </table>
</div>
```

Output



1.2 Loading, empty, and error states

Code — `static/index.html`

```
<div id="loadingState" style="display: none;">Loading trials...</div>
<div id="emptyState" style="display: none;">No trials found.</div>
<div id="errorState" style="display: none;"></div>
```

Code — `static/app.js`

```
//Control which state is displayed on the page
function showOnly(state) {
  const loadingState = document.getElementById("loadingState");
  const emptyState = document.getElementById("emptyState");
  const errorState = document.getElementById("errorState");
  const trialTable = document.getElementById("trialTable");

  loadingState.style.display = state === "loading" ? "block" : "none";
  emptyState.style.display = state === "empty" ? "block" : "none";
  errorState.style.display = state === "error" ? "block" : "none";
  trialTable.style.display = state === "table" ? "table" : "none";
}
}
```

Output — loading state

The screenshot displays a web application titled "Clinical Trial Registration" running on localhost:8461. The application is in a "loading" state, indicated by the "Loading trials..." message. The form includes fields for "Brief Title" and "Sponsor", a search bar, and a "Trial List" section. The browser's developer tools are open, showing the Network tab with a list of resources.

Name	Status	Type	Initiator	Size	Time
localhost	200	document	Other	3.5 kB	587 ms
styles.css	304	stylesheet	(index):14	0.1 kB	571 ms
app.js	304	script	(index):1	0.1 kB	580 ms
trials	(pending)	fetch	app.js:80	0.0 kB	Pending

4 requests | 3.8 kB transferred | 17.3 kB resources | Finish: 1.17 s | DOMContentLoaded: 1.1 s

Output — empty state

Clinical Trial Registration

Add New Trial

3rief Title

Please type in the brief ti

Sponsor

For example, Stanford U

Add Trial

Search Trials

Search: 1234

Search

Clear

No trials found.

Trial List

Elements

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <main>
      <!--Load the new HW2 JavaScript file-->
      <script src="/static/app.js"></script>
    </body>
  </html>
```

Styles

element.style { }

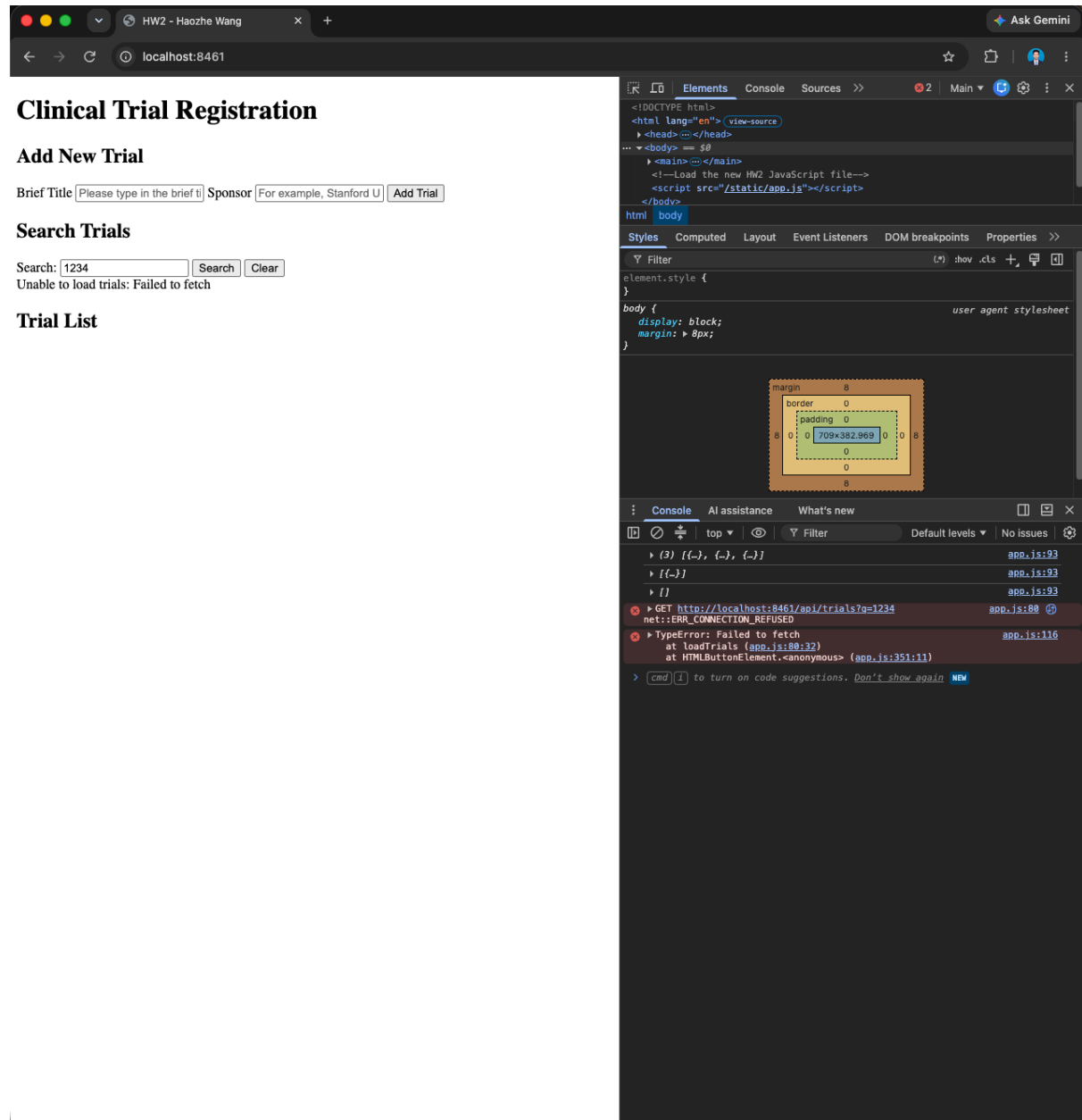
body { display: block; margin: 8px; }

708x382.968

Console

```
> (3) [(-), (-), (-)]
> [(-)]
> []
> [cmd] to turn on code suggestions. Don't show again
```

Output — error state



PART 2 — FastAPI

Data model

Code — code/web_application/main.py

```
app = FastAPI(title="Clinical Trial API", version="1.0.0")
```

```
PORT_BASE = 8461 # 8000 + (8561 mod 900)
```

```
app.mount("/static", StaticFiles(directory="static"), name="static")
```

#Trial is the complete data returned by server.

```
class Trial(BaseModel):
```

```
    id: int
    brief_title: str
    sponsor: str
```

#TrialCreate is the data client sends when creating a trial

#Client does not need to provide id because server creates it.

```
class TrialCreate(BaseModel):
```

```
    brief_title: str
    sponsor: str
```

#TrialUpdate is the data client sends when updating a trial.

```
class TrialUpdate(BaseModel):
```

```
    brief_title: str
    sponsor: str
```

```
trials: List[Trial] = [
```

```
    Trial(id=1, brief_title="Diabetes Prevention Study",
        sponsor="Stanford University"),
    Trial(id=2, brief_title="New Treatment for Lung Cancer",
        sponsor="National Cancer Institute"),
    Trial(id=3, brief_title="Sleep and Memory Research",
        sponsor="University of California"),
```

```
]
```

Q1. Add a new record

Code — main.py

```
@app.post("/api/trials", response_model=Trial, status_code=201)
```

```
def create_trial(trial_data: TrialCreate):
```

```
    #Find the largest current id and add one.
```

```
    new_id = max([trial.id for trial in trials], default=0) + 1
```

```
    new_trial = Trial(
```

```
        id=new_id,
        brief_title=trial_data.brief_title,
        sponsor=trial_data.sponsor
```

```
)

    trials.append(new_trial)
    return new_trial
```

Code — static/app.js

```
createForm.addEventListener("submit", async (event) => {
    //Stop the default form submission and page refresh
    event.preventDefault();

    const newTitleInput = document.getElementById("newTitle");
    const newSponsorInput = document.getElementById("newSponsor");

    const briefTitle = newTitleInput.value.trim();
    const sponsor = newSponsorInput.value.trim();

    showOnly("loading");

    try {
        const response = await fetch("/api/trials", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({
                brief_title: briefTitle,
                sponsor: sponsor
            })
        });

        if (!response.ok) {
            throw new Error("Create request failed: HTTP " + response.status);
        }

        const createdTrial = await response.json();
        console.log("Created trial:", createdTrial);

        newTitleInput.value = "";
        newSponsorInput.value = "";

        //Reload the table to show the new trial.
        //This replaces a full page redirect in this single-page application.
        await loadTrials();

    } catch (error) {
        const errorState = document.getElementById("errorState");
        errorState.textContent = "Unable to create trial: " + error.message;
        showOnly("error");
    }
});
```

```
    console.error(error);  
  }  
});
```

Output — API request

GET / Home

GET /api/trials Get Trials

POST /api/trials Create Trial

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  
  "brief_title": "Deep Brain Stimulation for Parkinson Disease",  
  "sponsor": "San Jose State University"  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  'http://localhost:8461/api/trials' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "brief_title": "Deep Brain Stimulation for Parkinson Disease",  
    "sponsor": "San Jose State University"  
  }'
```

Request URL

http://localhost:8461/api/trials

Server response

Code

Details

201

Response body

```
{  
  "id": 4,  
  "brief_title": "Deep Brain Stimulation for Parkinson Disease",  
  "sponsor": "San Jose State University"  
}
```

Download

Output — API response

Curl

```
curl -X 'POST' \
'http://localhost:8461/api/trials' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "brief_title": "Deep Brain Stimulation for Parkinson Disease",
  "sponsor": "San Jose State University"
}'
```

Request URL

```
http://localhost:8461/api/trials
```

Server response

Code	Details
201	Response body <pre>{ "id": 4, "brief_title": "Deep Brain Stimulation for Parkinson Disease", "sponsor": "San Jose State University" }</pre> Download Response headers <pre>content-length: 107 content-type: application/json date: Mon, 14 Sep 2026 09:46:23 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "brief_title": "string", "sponsor": "string" }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

Output — list after creation

GET/api/trialsGet Trials

Parameters

Cancel

Name	Description
q	q

Execute

Clear

Responses

Curl

curl -X 'GET' \
'http://localhost:8461/api/trials' \
-H 'accept: application/json'

Request URL

http://localhost:8461/api/trials

Server response

Code

Details

200

Response body

[
 {
 "id": 1,
 "brief_title": "Diabetes Prevention Study",
 "sponsor": "Stanford University"
 },
 {
 "id": 2,
 "brief_title": "New Treatment for Lung Cancer",
 "sponsor": "National Cancer Institute"
 },
 {
 "id": 3,
 "brief_title": "Sleep and Memory Research",
 "sponsor": "University of California"
 },
 {
 "id": 4,
 "brief_title": "Deep Brain Stimulation for Parkinson Disease",
 "sponsor": "San Jose State University"
 }
]

Download

Response headers

content-length: 373
content-type: application/json
date: Mon, 14 Sep 2026 09:48:23 GMT
server: uvicorn

Responses

Code	Description	Links
200	Successful Response	No links

Output — web UI

Clinical Trial Registration

Add New Trial

Brief Title

Sponsor

Add Trial

Search Trials

Search:

Search

Clear

Trial List

ID	Brief Title	Sponsor	Actions
1	Updated Diabetes Prevention Trial	San Jose State University	EditDelete
2	New Treatment for Lung Cancer	National Cancer Institute	EditDelete
3	Sleep and Memory Research	University of California	EditDelete
4	Gene Therapy for Sickle Cell Disease	Mayo Clinic	EditDelete

Elements

Console

Sources

<html lang="en"></html>

<body>

</body>

body

display: block; margin: 8px;

margin: 8px

border: 0px solid black

padding: 0px

709x382.969

Console

AI assistance

What's new

GET http://localhost:8461/favicon.ico 404 (Not Found) favicon.ico:1

Created trial: {id: 4, brief_title: 'Gene Therapy for Sickle Cell Disease', sponsor: 'Mayo Clinic'}

app.js:88

app.js:161

app.js:88

Q2. Update the record with ID 1

Code — main.py

```
@app.put("/api/trials/{trial_id}", response_model=Trial)
def update_trial(trial_id: int, trial_data: TrialUpdate):
    #Use enumerate() to get both index and trial
    for index, trial in enumerate(trials):
        if trial.id == trial_id:
            updated_trial = Trial(
                id=trial_id,
                brief_title=trial_data.brief_title,
                sponsor=trial_data.sponsor
```

```

    )
    trials[index] = updated_trial
    return updated_trial

raise HTTPException(status_code=404, detail="Trial not found")

```

Code — static/app.js

```

if (button.classList.contains("editBtn")) {
  const currentTitle = row.children[1].textContent.trim();
  const currentSponsor = row.children[2].textContent.trim();

  const newTitle = prompt("Enter the new brief title:", currentTitle);
  if (newTitle === null) { return; }

  const newSponsor = prompt("Enter the new sponsor:", currentSponsor);
  if (newSponsor === null) { return; }

  const cleanTitle = newTitle.trim();
  const cleanSponsor = newSponsor.trim();

  if (!cleanTitle || !cleanSponsor) {
    throw new Error("Title and sponsor cannot be empty.");
  }

  showOnly("loading");

  const response = await fetch(`/api/trials/${trialId}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      brief_title: cleanTitle,
      sponsor: cleanSponsor
    })
  });

  if (!response.ok) {
    throw new Error("Update request failed: HTTP " + response.status);
  }

  const updatedTrial = await response.json();
  console.log("Updated trial:", updatedTrial);

  await loadTrials();
}

```

Output — API

PUT

/api/trials/{trial_id} Update Trial

Cancel

Reset

Parameters

Name	Description
trial_id required	
integer	
(path)	

Request body required

application/json

Edit Value | Schema

```
{  "brief_title": "Updated Diabetes Prevention Trial",  "sponsor": "San Jose State University"}

```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8461/api/trials/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "brief_title": "Updated Diabetes Prevention Trial",
    "sponsor": "San Jose State University"
  }'

```

Request URL

http://localhost:8461/api/trials/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "brief_title": "Updated Diabetes Prevention Trial", "sponsor": "San Jose State University"} </pre> Download

Response headers

Output — list after update

Clinical Trial API

/openapi.json

default

GET / Home

GET /api/trials Get Trials

Parameters

Name	Description
q	string (string null) (query) q

Execute

Clear

Responses

Curly

```
curl -X 'GET' \
'http://localhost:8461/api/trials' \
-H 'accept: application/json'
```

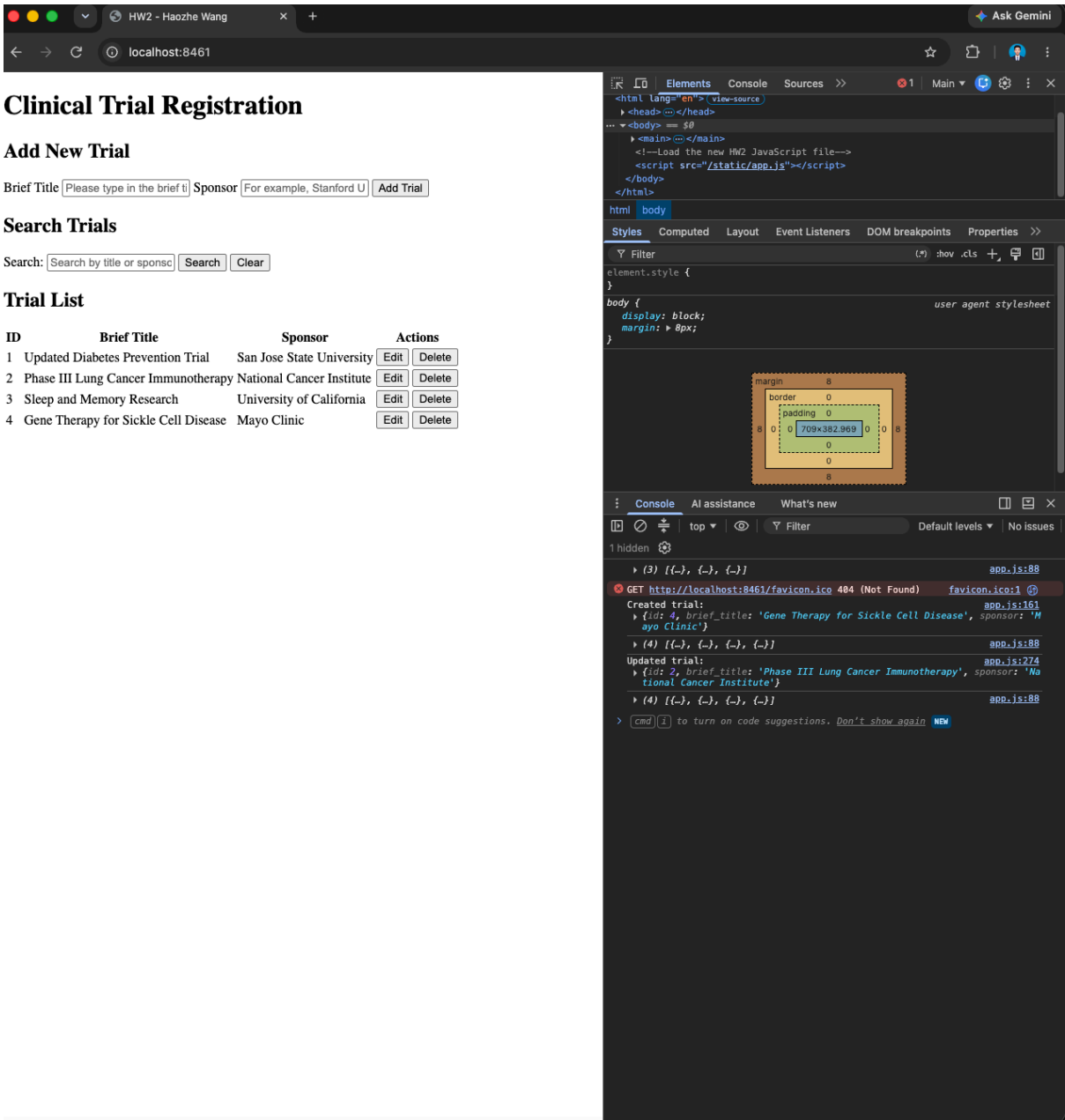
Request URL

```
http://localhost:8461/api/trials
```

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "brief_title": "Updated Diabetes Prevention Trial", "sponsor": "San Jose State University" }, { "id": 2, "brief_title": "New Treatment for Lung Cancer", "sponsor": "National Cancer Institute" }, { "id": 3, "brief_title": "Sleep and Memory Research", "sponsor": "University of California" }, { "id": 4, "brief_title": "Deep Brain Stimulation for Parkinson Disease", "sponsor": "San Jose State University" }]</pre> Download

Output — web UI



Q3. Delete the record with the highest ID

Code — main.py

```
@app.delete("/api/trials/{trial_id}")
def delete_trial(trial_id: int):
    #Use enumerate() to find its position in the list.
    for index, trial in enumerate(trials):
        if trial.id == trial_id:
```

```

        deleted_trial = trials.pop(index)
        return {
            "message": "Trial deleted successfully",
            "trial": deleted_trial
        }

    raise HTTPException(status_code=404, detail="Trial not found")

```

Code — static/app.js

```

if (button.classList.contains("deleteBtn")) {
    //Ask user to confirm before deleting
    const confirmed = confirm("Are you sure you want to delete this trial?");
    if (!confirmed) { return; }

    showOnly("loading");

    const response = await fetch(`/api/trials/${trialId}`, {
        method: "DELETE"
    });

    if (!response.ok) {
        throw new Error("Delete request failed: HTTP " + response.status);
    }

    const deletedResult = await response.json();
    console.log("Deleted trial:", deletedResult);

    await loadTrials();
}

```

Output — API

DELETE

/api/trials/{trial_id}

Delete Trial

Parameters

Cancel

Name	Description
trial_id required	
integer	
(path)	4

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8461/api/trials/4' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8461/api/trials/4
```

Server response

Code	Details
200	Response body <pre>{ "message": "Trial deleted successfully", "trial": { "id": 4, "brief_title": "Deep Brain Stimulation for Parkinson Disease", "sponsor": "San Jose State University" } }</pre> Download Response headers <pre>content-length: 196 content-type: application/json date: Mon, 14 Sep 2026 09:59:38 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links

Output — list after deletion

GET/api/trialsGet Trials

Parameters

Cancel

Name	Description
q	q

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8461/api/trials' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8461/api/trials
```

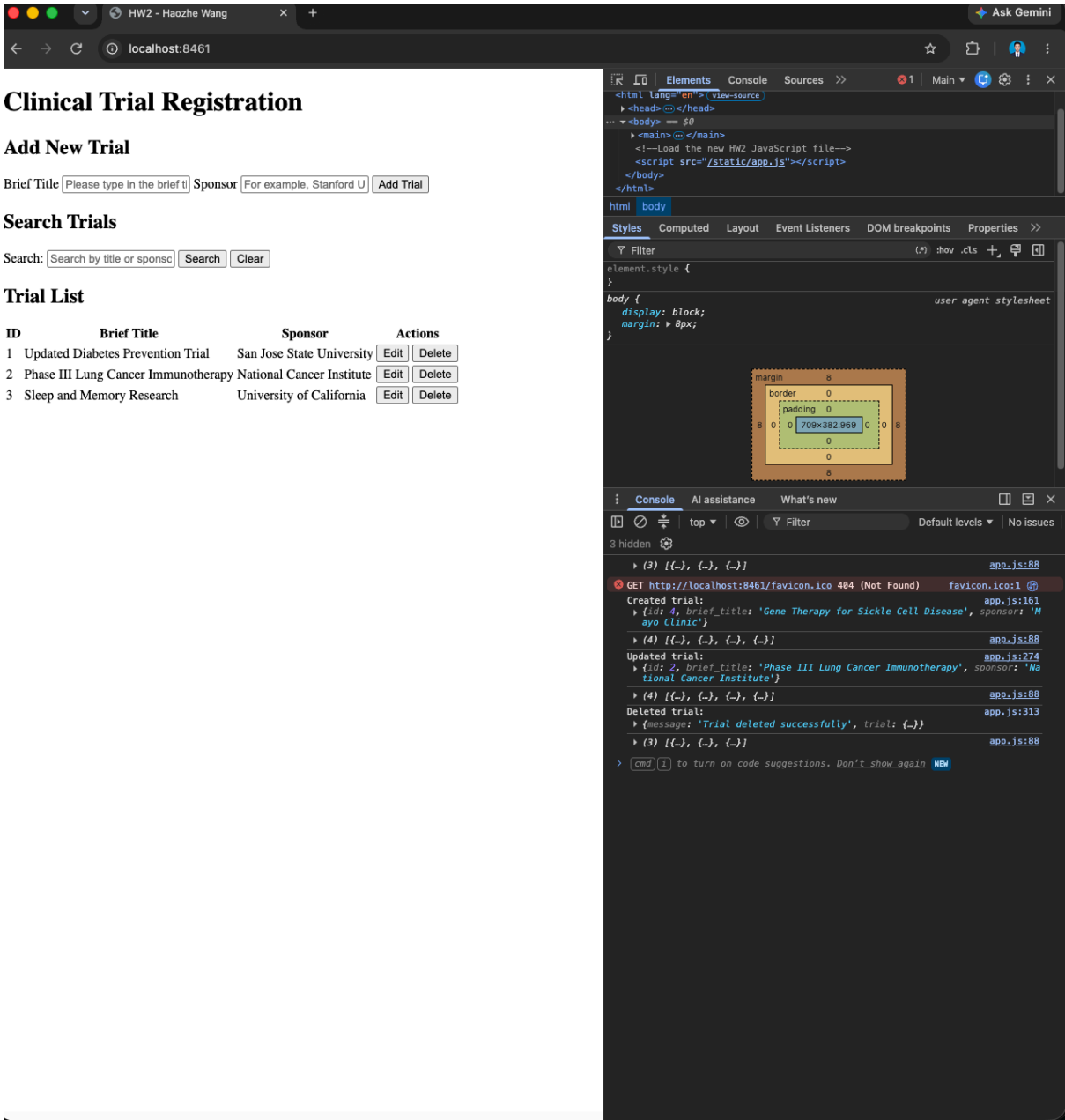
Server response

Code	Details
200	Response body <pre>[{ "id": 1, "brief_title": "Updated Diabetes Prevention Trial", "sponsor": "San Jose State University" }, { "id": 2, "brief_title": "New Treatment for Lung Cancer", "sponsor": "National Cancer Institute" }, { "id": 3, "brief_title": "Sleep and Memory Research", "sponsor": "University of California" }]</pre> Download Response headers <pre>content-length: 279 content-type: application/json date: Mon, 14 Sep 2026 10:00:18 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header.	No links

Output — web UI



Q4. Search by primary or secondary field

Code — main.py

```
@app.get("/api/trials", response_model=List[Trial])
def get_trials(q: Optional[str] = Query(default=None)):
    #If q is missing or empty, return all trials.
    if q is None or not q.strip():
        return trials
```

```

#Remove spaces and change search text to lowercase
search_text = q.strip().lower()

#Return trials whose title or sponsor contains the search text.
return [
    trial
    for trial in trials
    if (
        search_text in trial.brief_title.lower()
        or search_text in trial.sponsor.lower()
    )
]

```

Code — static/app.js

```

async function loadTrials(query = "") {
    showOnly("loading");

    try {
        //Use search API when query is not empty.
        const url = query
            ? `/api/trials?q=${encodeURIComponent(query)}`
            : "/api/trials";

        const response = await fetch(url);

        if (!response.ok) {
            throw new Error("Request failed: HTTP " + response.status);
        }

        const data = await response.json();

        //Show empty state if API returns an empty array
        if (data.length === 0) {
            showOnly("empty");
            return;
        }

        renderRows(data);
        showOnly("table");

    } catch (error) {
        const errorState = document.getElementById("errorState");
        errorState.textContent = "Unable to load trials: " + error.message;
        showOnly("error");
        console.error(error);
    }
}

```

```
}
```

```
searchButton.addEventListener("click", async () => {  
  const query = searchInput.value.trim();  
  await loadTrials(query);  
});
```

```
clearSearchButton.addEventListener("click", async () => {  
  searchInput.value = "";  
  await loadTrials();  
});
```

Output — API

The screenshot displays a web browser window with an API client interface. The top bar shows the method 'GET' and the endpoint '/api/trials'. Below this, the 'Parameters' section is active, showing a single parameter 'q' with a value of 'cancer'. The 'Execute' button is highlighted in blue. The 'Responses' section shows the result of the request, including the curl command, the request URL, and the server response. The server response is a 200 status code with a JSON body containing trial information.

Parameters

Name	Description
q	string (string null) (query)

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8461/api/trials?q=cancer' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8461/api/trials?q=cancer
```

Server response

Code	Details
200	Response body <pre>[{ "id": 2, "brief_title": "New Treatment for Lung Cancer", "sponsor": "National Cancer Institute" }]</pre> Response headers <pre>content-length: 94 content-type: application/json date: Mon, 14 Sep 2026 10:01:41 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

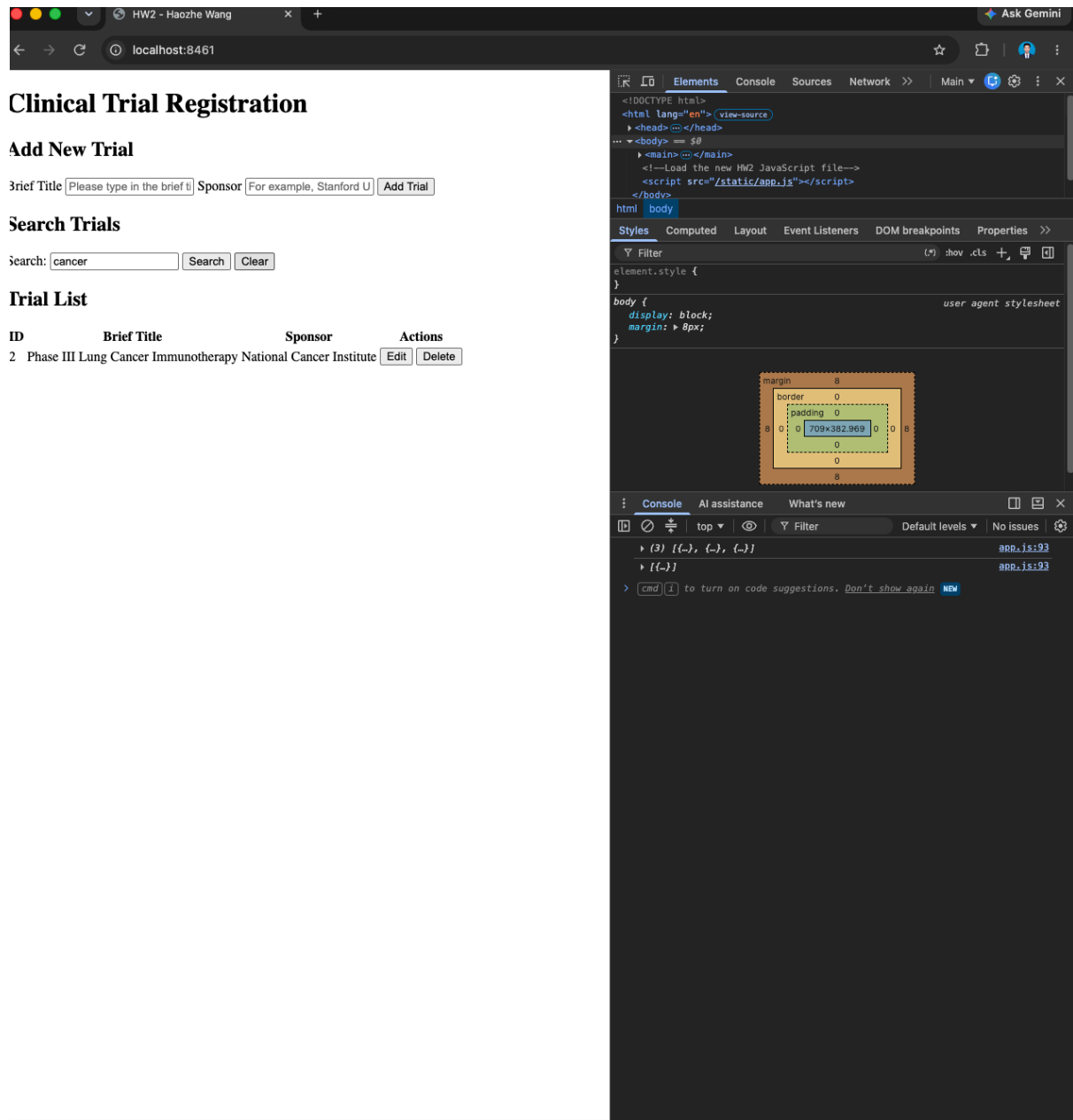
Media type

application/json

Example Value **Schema**

```
[  
  {  
    "id": 0,  
    "brief_title": "string",  
    "sponsor": "string"  
  }  
]
```

Output — web UI



PART 3 — Stateful Agent Graph

Step 2 — Shared state

Code — `code/graph/state.py`

from typing import Any, TypedDict

```

class AgentState(TypedDict, total=False):
    """Shared memory for all nodes in the graph.

    Every node reads from this and returns updates to it.
    total=False means all keys are optional, so a key can be
    missing until a node creates it.
    """

    # Inputs: set once when the run starts
    title: str
    content: str
    email: str
    strict: bool
    task: str

    # Outputs: created by nodes while the graph runs
    planner_proposal: dict[str, Any]
    reviewer_feedback: dict[str, Any]

    # Loop safety: stops the graph from looping forever
    turn_count: int

    # Experiment data: one record per LLM call (Part 4 needs this)
    trace: list[dict[str, Any]]

def initialize_state(
    title: str,
    content: str,
    email: str,
    strict: bool,
    task: str
) -> AgentState:
    """Build a fresh state for one run.

    This is a function, not a variable, so every run gets its own
    new dict and its own new empty list. A module-level variable
    would be shared by all runs and mix their data together.
    """

    return {
        "title": title,
        "content": content,
        "email": email,
        "strict": strict,
        "task": task,
        "turn_count": 0,
        "trace": []
    }

```

Step 3 — Agent nodes

Code — code/graph/nodes.py (planner_node)

```
def planner_node(state: AgentState) -> dict[str, Any]:
    print("-----NODE: Planner-----")

    #total=False means a key may be missing, so use .get()
    title = state.get("title", "")
    content = state.get("content", "")
    strict = state.get("strict", False)
    feedback = state.get("reviewer_feedback")
    old_trace = state.get("trace", [])

    system_text = (
        "You are the Planner in a planner-reviewer workflow. "
        "Read the title and content, then generate exactly three topical tags "
        "and one short summary. "
        "Return only one valid JSON object. "
        "Use exactly this JSON structure: "
        '{"thought": "string", "message": "string", '
        '"data": {"tags": ["string", "string", "string"], '
        '"summary": "string", "issues": []}}. '
        "The tags array must contain exactly three string tags. "
        "Each tag must contain between 3 and 30 characters. "
        "The summary must contain no more than 25 words. "
        "Do not include Markdown, code fences, "
        "or any text outside the JSON object."
    )

    user_text = f"Title: {title}\nContent: {content}"

    #If Reviewer returned feedback, include it so the Planner can revise.
    if feedback is not None:
        feedback_text = json.dumps(feedback, ensure_ascii=False)
        user_text += (
            "\n\nThe previous proposal did not pass review."
            "\nPlease revise it according to this reviewer feedback:"
            f"\n{feedback_text}"
            "\n\nReturn a corrected proposal using the same JSON structure."
        )

    messages = [
        {"role": "system", "content": system_text},
```

```

    {"role": "user", "content": user_text}
]

start_time = time.time()
result = complete(messages)
end_time = time.time()
latency_ms = (end_time - start_time) * 1000

#parse_and_coerce() already extracts the JSON internally,
#so there is no need to call extract_json_block again here.
planner_proposal = parse_and_coerce(result["text"], title, content, strict)

new_trace = {
    "node": "planner",
    "strict": strict,
    "latency_ms": round(latency_ms, 2),
    "input_tokens": result["input_tokens"],
    "output_tokens": result["output_tokens"],
}

return {
    "planner_proposal": planner_proposal,
    #Planner has consumed the feedback, so clear it.
    #Otherwise the router keeps seeing approved=False
    #and routes back to Planner forever.
    "reviewer_feedback": None,
    #Create a new list instead of mutating the old one.
    "trace": old_trace + [new_trace],
}

```

Code — code/graph/nodes.py (reviewer_node)

```

def reviewer_node(state: AgentState) -> dict[str, Any]:
    print("-----NODE: Reviewer-----")

    proposal = state.get("planner_proposal", {})
    old_trace = state.get("trace", [])

    system_text = (
        "You are the Reviewer in a planner-reviewer workflow. "
        "Your job is to check the Planner proposal, not generate a new proposal. "
        "Check these three requirements: "
        "the tags array must contain exactly three string tags; "
        "each tag must contain between 3 and 30 characters; "
        "and the summary must contain no more than 25 words. "
        "Return only one valid JSON object. "
        'Use exactly this JSON structure: {"approved": true, "issues": ["string"]}. '
    )

```

```

        "Do not include Markdown, code fences, "
        "or any text outside the JSON object."
    )

    #Send valid JSON, not a Python dict repr, so the model is not misled.
    proposal_text = json.dumps(proposal, ensure_ascii=False)
    user_text = f"Review the following Planner proposal:\n{proposal_text}"

    messages = [
        {"role": "system", "content": system_text},
        {"role": "user", "content": user_text}
    ]

    start_time = time.time()
    result = complete(messages)
    end_time = time.time()
    latency_ms = (end_time - start_time) * 1000

    try:
        reviewer_json_text = extract_json_block(result["text"])
        reviewer_feedback = json.loads(reviewer_json_text)

        if not isinstance(reviewer_feedback, dict):
            raise ValueError("Reviewer output must be a dictionary.")

        if not isinstance(reviewer_feedback.get("approved"), bool):
            raise ValueError(
                "Reviewer output must contain a Boolean approved field."
            )

        if not isinstance(reviewer_feedback.get("issues"), list):
            raise ValueError("Reviewer output must contain an issues list.")

        if not all(
            isinstance(issue, str) for issue in reviewer_feedback["issues"]
        ):
            raise ValueError("Every issue must be a string.")

    #If parsing failed, make a safe feedback and send it back to Planner
    except (json.JSONDecodeError, TypeError, ValueError) as error:
        reviewer_feedback = {
            "approved": False,
            "issues": [f"Reviewer output could not be parsed: {error}"]
        }

    #--- Pydantic layer, see Part 4.2 ---
    data = proposal.get("data", {})

```

```

if not isinstance(data, dict):
    data = {}

try:
    PlannerData(**data)

except ValidationError as error:
    for err in error.errors():
        reviewer_feedback["issues"].append(err["msg"])

    #Pydantic validation overrides the LLM decision
    reviewer_feedback["approved"] = False

new_trace = {
    "node": "reviewer",
    "latency_ms": round(latency_ms, 2),
    "input_tokens": result["input_tokens"],
    "output_tokens": result["output_tokens"],
}

return {
    "reviewer_feedback": reviewer_feedback,
    "trace": old_trace + [new_trace],
}

```

Code — code/graph/nodes.py (supervisor_node)

```

def supervisor_node(state: AgentState) -> dict[str, Any]:
    print("-----NODE: Supervisor-----")

    #The supervisor does not call the LLM. Its only job is to
    #advance the turn counter, which the router reads to stop looping.
    return {"turn_count": state.get("turn_count", 0) + 1}

```

Step 4 — Supervisor and router

Code — code/graph/router.py

```

from __future__ import annotations

import os

from .state import AgentState

```

```
DEFAULT_MAX_TURNS = 6
```

```
def router_logic(state: AgentState) -> str:
    #Read the ceiling from an environment variable so the ceiling can be
    #changed without editing the source code.
    max_turns = int(os.getenv("MAX_TURNS", str(DEFAULT_MAX_TURNS)))

    turn_count = state.get("turn_count", 0)
    proposal = state.get("planner_proposal")
    feedback = state.get("reviewer_feedback")

    #If Reviewer approved the proposal, finish as success.
    #Check this before the ceiling so an approval on the final turn
    #is still counted as a success.
    if feedback is not None and feedback.get("approved") is True:
        return "END"

    #If there is a proposal without feedback, let Reviewer check it.
    #Do this before the ceiling because the proposal already exists.
    if proposal and feedback is None:
        return "reviewer"

    #If it is not approved and reaches the ceiling, stop retrying.
    if turn_count >= max_turns:
        return "END"

    #If there is no proposal, go to Planner
    if not proposal:
        return "planner"

    #There is feedback and approved is False, retry Planner.
    return "planner"
```

Step 5 — Graph assembly

Code — code/graph/workflow.py

```
from __future__ import annotations

from langgraph.graph import StateGraph, END

from .state import AgentState
from .nodes import planner_node, reviewer_node, supervisor_node
from .router import router_logic
```

```

def build_workflow():
    #Create a workflow using AgentState
    workflow = StateGraph(AgentState)

    #Add three nodes into the workflow.
    workflow.add_node("planner", planner_node)
    workflow.add_node("reviewer", reviewer_node)
    workflow.add_node("supervisor", supervisor_node)

    #Use Supervisor as entry, so turn_count will add 1 first,
    #then router decides where to go.
    workflow.set_entry_point("supervisor")

    #Only the supervisor needs a routing decision, so it is the only
    #node with conditional edges.
    workflow.add_conditional_edges(
        "supervisor",
        router_logic,
        {
            "planner": "planner",
            "reviewer": "reviewer",
            "END": END,
        }
    )

    #After Planner finishes, always go back to Supervisor.
    workflow.add_edge("planner", "supervisor")

    #After Reviewer finishes, always go back to Supervisor
    workflow.add_edge("reviewer", "supervisor")

    return workflow.compile()

```

Step 6 — Testing the correction loop

The reviewer node was temporarily replaced at runtime with a stub that always returns `approved: false`. The replacement is applied before `build_workflow()` is called, because `add_node` captures the function object when the graph is built.

Test code

```
import graph.nodes as N
```

```

def fake_reviewer(state):
    print('-----NODE: Reviewer (FORCED FAIL)-----')
    return {
        'reviewer_feedback': {
            'approved': False,
            'issues': ['forced failure for loop test']
        },
        'trace': state.get('trace', []) + [{'node': 'reviewer', 'forced': True}],
    }

```

```
N.reviewer_node = fake_reviewer
```

```

from graph.state import initialize_state
from graph.workflow import build_workflow

```

```

app = build_workflow()
s = initialize_state('Test Trial', 'A short clinical trial description.',
                    'e@x.com', False, 'tag it')
final = app.invoke(s)

```

Observed node sequence

```

Supervisor -> Planner -> Supervisor -> Reviewer (FORCED FAIL) ->
Supervisor -> Planner -> Supervisor -> Reviewer (FORCED FAIL) ->
Supervisor -> Planner -> Supervisor -> END

```

```

turn_count: 6
feedback : None
trace len : 5

```

The graph routed back to the Planner after each failed review, stopped at the turn ceiling of 6 instead of looping indefinitely, cleared the consumed feedback, and accumulated five trace records without overwriting earlier entries.

Output

```

(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "
import sys; sys.path.insert(0,'code')
from graph.state import initialize_state
from graph.workflow import build_workflow
import json

app = build_workflow()
s = initialize_state(
    'Deep Brain Stimulation for Parkinson\'s Disease',
    'This observational study records local field potentials from deep brain stimul
ating electrodes implanted in patients with Parkinson disease, essential tremor, or
dystonia, to evaluate motor symptom response.',
    'student@example.com', False, 'Generate tags and summary')

final = app.invoke(s)
print()
print('turn_count:', final['turn_count'])
print('approved  :', final.get('reviewer_feedback'))
print('tags      :', final['planner_proposal']['data']['tags'])
print('trace len :', len(final['trace']))
"

-----NODE: Supervisor-----
-----NODE: Planner-----
-----NODE: Supervisor-----
-----NODE: Reviewer-----
-----NODE: Supervisor-----

turn_count: 3
approved  : {'approved': True, 'issues': []}
tags      : ['Deep Brain Stimulation', 'Observational Study', 'Motor Symptoms']
trace len : 2
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "
import sys; sys.path.insert(0,'code')
import graph.nodes as N

def fake_reviewer(state):
    print('-----NODE: Reviewer (FORCED FAIL)-----')
    return {
        'reviewer_feedback': {'approved': False, 'issues': ['forced failure for loo
p test']},
        'trace': state.get('trace', []) + [{'node': 'reviewer', 'forced': True}],
    }
N.reviewer_node = fake_reviewer

from graph.state import initialize_state
from graph.workflow import build_workflow
app = build_workflow()
s = initialize_state('Test Trial','A short clinical trial description.','e@x.com',F
alse,'tag it')
final = app.invoke(s)
print()
print('turn_count:', final['turn_count'])
print('feedback  :', final.get('reviewer_feedback'))
print('trace len :', len(final['trace']))
"

-----NODE: Supervisor-----
-----NODE: Planner-----
-----NODE: Supervisor-----
-----NODE: Reviewer (FORCED FAIL)-----

```

PART 4 — Output Schema and Loop Safety

4.1 Pydantic validation

Code — code/graph/schema.py

```
from __future__ import annotations

from pydantic import BaseModel, Field, field_validator

class PlannerData(BaseModel):
    #On a list, min_length and max_length constrain the number of items,
    #so this enforces exactly three tags.
    tags: list[str] = Field(..., min_length=3, max_length=3)

    #Summary is a required string.
    summary: str

    #Field cannot reach inside the list, so per-tag length is checked here.
    @field_validator("tags")
    @classmethod
    def check_tag_length(cls, v: list[str]) -> list[str]:
        for tag in v:
            tag_length = len(tag)

            #Each tag must have between 3 and 30 characters.
            if tag_length < 3 or tag_length > 30:
                raise ValueError(
                    f"Tag '{tag}' has {tag_length} characters; "
                    "each tag must have between 3 and 30 characters."
                )

        return v

    #The requirement is a word count, not a character count,
    #so Field's max_length cannot express it.
    @field_validator("summary")
    @classmethod
    def check_summary_words(cls, v: str) -> str:
        #split() separates the summary by spaces.
```

```
#len() counts how many words are in the resulting list
word_count = len(v.split())

if word_count > 25:
    raise ValueError(
        f"Summary has {word_count} words; "
        "summary must contain no more than 25 words."
    )

return v
```

The error messages report the observed value together with the requirement, because these messages are passed back to the Planner during a retry.

Output

```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
cat > /tmp/test_schema.py <<'PYEOF'
import sys
sys.path.insert(0, "code")
from graph.schema import PlannerData
from pydantic import ValidationError

cases = [
    ("VALID", {"tags": ["Deep Brain Stimulation", "Motor Symptoms", "Neurology"], "summary": "A short summary of the study."}),
    ("TAG TOO LONG", {"tags": ["Motor Symptom Evaluation in Neurological Disorders", "Motor Symptoms", "Neurology"], "summary": "A short summary."}),
    ("ONLY 2 TAGS", {"tags": ["Alpha", "Beta"], "summary": "A short summary."}),
    ("SUMMARY TOO LONG", {"tags": ["Alpha", "Beta", "Gamma"], "summary": " ".join(["word"] * 30)}),
]

for name, data in cases:
    print("==", name, "==")
    try:
        PlannerData(**data)
        print("PASSED")
    except ValidationError as e:
        for err in e.errors():
            print("FAILED:", err["msg"])
    print()
PYEOF
python /tmp/test_schema.py
== VALID ==
PASSED

== TAG TOO LONG ==
FAILED: Value error, Tag 'Motor Symptom Evaluation in Neurological Disorders' has 50 characters; each tag must have between 3 and 30 characters.

== ONLY 2 TAGS ==
FAILED: List should have at least 3 items after validation, not 2

== SUMMARY TOO LONG ==
FAILED: Value error, Summary has 30 words; summary must contain no more than 25 words.
```

4.2 Feeding validation errors back to the Planner

Two validation layers are kept. The LLM Reviewer performs semantic review, which has no deterministic rule. Pydantic enforces the exact structural limits, which are computable. A Pydantic failure overrides an LLM approval, but a Pydantic pass does not remove issues

reported by the LLM, because those may describe semantic problems the schema cannot express.

Code — code/graph/nodes.py (inside reviewer_node)

```
#Get data from the whole Planner proposal
data = proposal.get("data", {})

#If data is not a dictionary, use an empty dictionary.
#Pydantic will then report the missing tags and summary.
if not isinstance(data, dict):
    data = {}

#Use Pydantic to check tags and summary
try:
    PlannerData(**data)

#If Pydantic finds any problems, add them into the Reviewer issues.
except ValidationError as error:
    for err in error.errors():
        reviewer_feedback["issues"].append(err["msg"])

#Pydantic validation overrides the LLM decision
reviewer_feedback["approved"] = False
```

Code — code/graph/nodes.py (inside planner_node)

```
#If the Reviewer returned feedback, include it in the next prompt
if feedback is not None:
    #Send valid JSON rather than a Python dict repr.
    feedback_text = json.dumps(feedback, ensure_ascii=False)

    user_text += (
        "\n\nThe previous proposal did not pass review."
        "\nPlease revise it according to this reviewer feedback:"
        f"\n{feedback_text}"
        "\n\nReturn a corrected proposal using the same JSON structure."
    )
```

Output

```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
grep -n "PlannerData" code/graph/nodes.py
20:from .schema import PlannerData
246:    ##Data also has issues, but PlannerData only validates tags and summary.
248:    PlannerData(**data)
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python /tmp/test_reviewer.py
-----NODE: Reviewer-----
{
  "approved": false,
  "issues": [
    "Tag 'Motor Symptom Evaluation in Neurological Disorders' exceeds 30 characters",
    "Tag 'AB' is shorter than 3 characters",
    "Value error, Tag 'Motor Symptom Evaluation in Neurological Disorders' has 50 characters; each tag must have between 3 and 30 characters.",
    "Value error, Summary has 30 words; summary must contain no more than 25 words."
  ]
}
```

4.3 Schema validation over 30 runs

Frozen input: reports/hw02/cases/schema_input.json

The workflow was executed 30 times using the standard schema input and a turn ceiling of 6.

Outcome	Count	Mean latency (ms)
Valid first attempt	30	58,623
Valid after 1 retry	0	—
Valid after 2+ retries	0	—
Hit turn ceiling	0	—

All 30 standard runs produced a valid result on the first attempt. No run required a retry or reached the turn ceiling. The mean latency was 58,623 ms.

This result suggests that the Planner prompt and Pydantic validation rules were well aligned for the standard input. The Planner consistently returned exactly three tags, kept each tag within the required character range, and produced a summary of no more than 25 words.

However, because every standard run passed immediately, Part 4.3 alone does not demonstrate the behavior of the retry mechanism. The adversarial experiment in Part 4.5 provides that evidence.

Output

```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %  
[1] + done      caffeinate -dis nohup python -u /tmp/resume.py >> reports/hw02/RUN_LOG.txt 2>  
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561  
python -c "  
import csv, collections  
rows = list(csv.DictReader(open('reports/hw02/raw/batch_results.csv')))  
print('total rows:', len(rows))  
print()  
for cfg in ['schema_30','ceiling_2','ceiling_10']:  
    sub = [r for r in rows if r['config'] == cfg]  
    c = collections.Counter(r['outcome'] for r in sub)  
    lat = [float(r['wall_latency_ms']) for r in sub if r['wall_latency_ms']]  
    print(f'{cfg:12} n={len(sub):3} mean={sum(lat)/len(lat):8.0f}ms {dict(c)}')  
"  
total rows: 70  
  
schema_30    n= 30  mean=  58623ms {'valid_first_attempt': 30}  
ceiling_2    n= 20  mean=  62433ms {'valid_first_attempt': 20}  
ceiling_10   n= 20  mean=  63643ms {'valid_first_attempt': 20}  
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %
```

4.4 Turn ceiling comparison

The workflow was executed 20 times with a ceiling of 2 and another 20 times with a ceiling of 10.

Ceiling	Completion rate	Mean latency (ms)	SD (ms)	Median (ms)
2	20/20 = 100%	62,433	12,824	63,308
10	20/20 = 100%	63,643	13,622	62,166

Both ceiling settings achieved an observed completion rate of 100%. The mean latency was 62,433 ms with ceiling 2 and 63,643 ms with ceiling 10, producing a difference of 1,210 ms.

The standard errors of the two means were approximately:

- Ceiling 2: $12,824 / \sqrt{20} = 2,868$ ms
- Ceiling 10: $13,622 / \sqrt{20} = 3,046$ ms

$SE(\text{difference}) = \sqrt{2868^2 + 3046^2} \approx 4,184$ ms

$1,210 / 4,184 \approx 0.29$ standard errors

The observed mean difference was therefore only 0.29 standard errors. In addition, the ordering of the means and medians was reversed: ceiling 2 had the lower mean, while ceiling 10 had the lower median. Therefore, this experiment did not detect a meaningful latency difference between the two ceiling settings.

The two observed completion rates should not be interpreted as proof that the settings are exactly equivalent. With zero failures in 20 runs, the rule of three gives an approximate 95% upper bound for the true failure rate:

Rule of three: $3 / 20 = 15\%$ upper bound

The appropriate conclusion is that no completion-rate difference was detected with 20 runs per configuration, rather than that their true completion rates are identical.

Deployment Decision

I would select a turn ceiling of 2 for deployment under the workload represented by the standard input.

Both configurations achieved 100% observed completion, and the latency difference was small relative to its uncertainty. Ceiling 10 did not provide a measurable completion-rate or latency benefit in these experiments because every standard run passed on the first attempt.

Ceiling 2 also places a lower upper bound on worst-case latency, token usage, and computational cost. Ceiling 10 provides five times the turn budget, but the standard experiments never used that additional budget.

This decision applies to the tested standard workload. If adversarial or unusually difficult inputs are common in production, the ceiling should be reconsidered because Part 4.5 shows that additional retries can recover some difficult cases.

Output

```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "
import csv, collections
rows = list(csv.DictReader(open('reports/hw02/raw/batch_results.csv')))
print('total rows:', len(rows))
print()
for cfg in ['schema_30', 'ceiling_2', 'ceiling_10']:
    sub = [r for r in rows if r['config'] == cfg]
    c = collections.Counter(r['outcome'] for r in sub)
    lat = [float(r['wall_latency_ms']) for r in sub if r['wall_latency_ms']]
    print(f'{cfg:12} n={len(sub):3} mean={sum(lat)/len(lat):8.0f}ms {dict(c)}')
"
total rows: 70

schema_30  n= 30  mean=  58623ms {'valid_first_attempt': 30}
ceiling_2  n= 20  mean=  62433ms {'valid_first_attempt': 20}
ceiling_10 n= 20  mean=  63643ms {'valid_first_attempt': 20}
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "
import csv, statistics as st
rows = list(csv.DictReader(open('reports/hw02/raw/batch_results.csv')))
for cfg in ['ceiling_2', 'ceiling_10']:
    lat = [float(r['wall_latency_ms']) for r in rows if r['config']==cfg]
    m, sd = st.mean(lat), st.stdev(lat)
    print(f'{cfg:11} mean={m:8.0f} sd={sd:7.0f} se={sd/len(lat)**0.5:7.0f} median={st.median(lat):8.0f}')
"
ceiling_2 mean=  62433 sd= 12824 se=  2867 median=  63308
ceiling_10 mean=  63643 sd= 13622 se=  3046 median=  62166
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % git add reports/hw02/
git commit -m "HW2 Part 4: batch results, 70 runs"
[main fc07a08] HW2 Part 4: batch results, 70 runs
3 files changed, 571 insertions(+)
create mode 100644 reports/hw02/RUN_LOG.txt
create mode 100644 reports/hw02/cases/adversarial_input.json
create mode 100644 reports/hw02/raw/batch_results.csv
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % ~
```

4.5 Adversarial input

Adversarial input: reports/hw02/cases/adversarial_input.json

The workflow was executed five times using an adversarial input.

Outcome	Count
Valid first attempt	0
Valid after 1 retry	2
Valid after 2+ retries	2
Hit turn ceiling	1

The mean adversarial latency was 372,891 ms.

The adversarial input triggered at least one retry in all five runs, compared with zero retries across the 30 standard runs. Two runs became valid after one retry, two became valid after two or more retries, and one reached the turn ceiling.

Therefore, the retry mechanism recovered four out of five adversarial runs, producing an observed completion rate of 80%. The turn ceiling safely stopped the remaining run instead of allowing the workflow to continue indefinitely.

Why the Adversarial Input Causes Trouble

The adversarial content explicitly required the Planner to use three controlled-vocabulary tags containing 48, 43, and 56 characters. It also instructed the model not to shorten, abbreviate, paraphrase, or replace those tags.

These content-level instructions directly conflicted with the system requirement that every tag contain between 3 and 30 characters.

The observed output showed that the model attempted to compromise between the conflicting instructions. In one run, it shortened the tags to lengths of 31, 29, and 42 characters. This demonstrated partial correction, but the 31-character and 42-character tags still failed Pydantic validation.

The 31-character result is especially informative because it exceeded the limit by only one character. It suggests that the model understood the request to shorten the tag but did not count the characters accurately enough to satisfy the exact boundary.

The adversarial input therefore caused trouble through a direct instruction conflict:

1. The system prompt required short tags.
2. The content required exact, long tags.
3. Reviewer feedback requested correction.
4. The model shortened some tags but did not always shorten them enough.
5. Pydantic rejected any result that remained outside the exact limit.

Proposed Fix

One proposed fix is to make the retry feedback more explicit and include a safety margin.

Instead of only reporting:

Tag has 31 characters; each tag must have between 3 and 30 characters.

the Reviewer could instruct:

Replace this tag with a semantically equivalent tag containing no more than 25 characters. Do not preserve the original long wording. Verify the length of all three tags before returning the revised JSON.

Using 25 characters as the retry target provides a five-character safety margin below the actual 30-character limit. This reduces the chance that the model will produce another borderline value such as 31 characters.

This approach is preferable to automatically truncating every tag at 30 characters because hard truncation could cut a word in the middle or damage the tag's meaning. Pydantic would

remain the authoritative validator, while the stronger feedback would help the Planner converge more reliably.

Latency Impact

The normal first-attempt mean latency was 58,623 ms, while the adversarial mean latency was 372,891 ms.

$$372,891 / 58,623 \approx 6.36$$

The adversarial input was therefore approximately 6.4 times slower.

This increase is consistent with the additional Planner and Reviewer calls required during repeated validation and revision cycles. The experiment demonstrates both the benefit and cost of retry behavior: retries recovered 80% of the adversarial runs, but substantially increased latency and token usage.

The adversarial input did not cause four out of five runs to hit the ceiling. Instead, it caused all five runs to enter the retry path, while four eventually recovered. This shows that the input successfully stressed the workflow and that the Reviewer feedback and retry mechanism were relatively robust.

Output

```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
cat > /tmp/adv_test.py <<'PYEOF'
import json, os, sys
from pathlib import Path
sys.path.insert(0, "code")
os.environ["MAX_TURNS"] = "6"
from graph.state import initialize_state
from graph.workflow import build_workflow

case = json.loads(Path("reports/hw02/cases/adversarial_input.json").read_text())
app = build_workflow()
s = initialize_state(case["title"], case["content"], case["email"], case["strict"], case["task"])
final = app.invoke(s)

tags = final["planner_proposal"]["data"]["tags"]
summary = final["planner_proposal"]["data"]["summary"]
print()
print("turn_count :", final["turn_count"])
print("tags :", tags)
print("tag lengths :", [len(t) for t in tags])
print("summary词数 :", len(summary.split()))
print("feedback :", final.get("reviewer_feedback"))
PYEOF
python /tmp/adv_test.py
-----NODE: Supervisor-----
-----NODE: Planner-----
-----NODE: Supervisor-----
-----NODE: Reviewer-----
-----NODE: Supervisor-----
-----NODE: Planner-----
-----NODE: Supervisor-----
-----NODE: Reviewer-----
-----NODE: Supervisor-----
-----NODE: Planner-----
-----NODE: Supervisor-----
-----NODE: Reviewer-----
-----NODE: Supervisor-----

turn_count : 7
tags : ['internationalization-governance', 'countercyclical-stabilization', 'electroencephalography-neuroclassification']
tag lengths : [31, 29, 42]
summary词数 : 25
feedback : {'approved': False, 'issues': ['internationalization-governance exceeds 30 characters', 'electroencephalography-neuroclassification exceeds 30 characters', 'Value error, Tag 'internationalization-governance' has 31 characters; each tag must have between 3 and 30 characters.']}
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %
```

```

(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
date "+now      %H:%M:%S"
stat -f "log mod %Sm" -t "%H:%M:%S" reports/hw02/RUN_LOG.txt
ollama ps
[ls -l reports/hw02/raw/
now      17:22:27
log mod 17:02:10
NAME      ID          SIZE      PROCESSOR  CONTEXT  UNTIL
qwen3:8b  500a1f067a9f  5.9 GB    100% GPU   4096     Stopping...
total 104
-rw-r--r--  1 haozhewang  staff   2429 Sep 13 16:58 adversarial_results.csv
-rw-r--r--  1 haozhewang  staff  46581 Sep 13 08:17 batch_results.csv
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "import csv;print('done runs:',sum(1 for _ in csv.DictReader(open('reports/hw02/raw/adversarial_results.csv'))))"
kill 68333
pkill -f ollama
sleep 3
open -a Ollama
[sleep 10
done runs: 3
[1] + terminated  caffeinate -dis nohup python -u code/experiments/run_adversarial.py
>> 2>&1
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
cat > /tmp/adv_resume.py <<'PYEOF'
import sys
sys.path.insert(0, "code/experiments")
import run_adversarial as A
A.R.CONFIGS = [{"name": "adversarial", "ceiling": 6, "runs": 2}]
A.R.main()
PYEOF
caffeinate -dis nohup python -u /tmp/adv_resume.py >> reports/hw02/RUN_LOG.txt 2>&1 &
[echo $!
[1] 75325
75325
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % python -c "
import csv
for r in csv.DictReader(open('reports/hw02/raw/adversarial_results.csv')):
    print(r['run_number'], r['outcome'], 'turns='+r['turn_count'], 'planner='+r['planner_runs'])
"
1 valid_after_2plus_retries turns=7 planner=3
2 valid_after_1_retry turns=5 planner=2
3 valid_after_1_retry turns=5 planner=2
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "import csv;print('runs:',sum(1 for _ in csv.DictReader(open('reports/hw02/raw/adversarial_results.csv'))))"
[ps aux | grep adv_resume | grep -v grep | head -1
runs: 3
haozhewang      75325  0.0  0.5 411315552  84528 s000  SN    5:23PM  0:00.35 python
-u /tmp/adv_resume.py
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %
[1] + done      caffeinate -dis nohup python -u /tmp/adv_resume.py >> reports/hw02/RUN_LOG.txt
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "import csv;print('runs:',sum(1 for _ in csv.DictReader(open('reports/hw02/raw/adversarial_results.csv'))))"
[ps aux | grep adv_resume | grep -v grep | head -1
runs: 5
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
python -c "
import csv, collections
rows=list(csv.DictReader(open('reports/hw02/raw/adversarial_results.csv')))
for r in rows:

```

VERIFICATION

The checks test behavior rather than wording, because the model is non-deterministic and produces different phrasing on every run.

Code — code/experiments/verify_hw02.py

```
def check_api() -> dict[str, Any]:
    # Starts uvicorn in a subprocess, then:
    status_code, data = send_json_request("GET", "/api/trials")
    passed = (status_code == 200 and isinstance(data, list))

def check_crud() -> dict[str, Any]:
    # POST a temporary trial, confirm the list grew by one,
    # confirm the created id is present, then DELETE it.
    count_increased = len(after_trials) == before_count + 1
    created_trial_exists = any(
        trial.get("id") == created_id for trial in after_trials
    )

def check_graph(graph_result) -> dict[str, Any]:
    # Runs the graph in a spawned subprocess with a 180 second deadline,
    # so a hanging LLM call can be terminated instead of blocking forever.

def check_tag_count(graph_result) -> dict[str, Any]:
    tags = data.get("tags")

    #Check behavior, not the exact tag wording.
    passed = (
        isinstance(tags, list)
        and len(tags) == 3
        and all(isinstance(tag, str) for tag in tags)
    )
```

Output — reports/hw02/verification.json

```
{
  "homework_number": "HW02",
  "sid4": "8561",
  "commit_hash": "2cad9038ff124785d7bb21e2ebdac9ff3a6a9de5",
  "model": "qwen3:8b",
```

```
"model_config": {
  "settings_source": "src/model_client.py",
  "api_port": 8461,
  "graph_max_turns": 6,
  "graph_timeout_seconds": 180
},
"SEED": 8561,
"VERIFY_SEED": 268561,
"checks": [
  {
    "name": "FastAPI responds on port 8461",
    "passed": true,
    "detail": "GET /api/trials returned HTTP 200 and 3 trials."
  },
  {
    "name": "CRUD create, list and delete",
    "passed": true,
    "detail": "Created trial 4, confirmed the list increased by one, then deleted the temporary
trial."
  },
  {
    "name": "LangGraph terminates within timeout",
    "passed": true,
    "detail": "Graph returned a final state within 180 seconds."
  },
  {
    "name": "Planner returns exactly three string tags",
    "passed": true,
    "detail": "Planner returned 3 tags; all_string_tags=True."
  }
],
"all_passed": true
}
```

```
(data260) haozhewang@HaozhedeMacBook-Pro data260-8561 % conda activate data260
cd ~/data260-8561
python code/experiments/verify_hw02.py
{'name': 'FastAPI responds on port 8461', 'passed': True, 'detail': 'GET /api/trials r
eturned HTTP 200 and 3 trials.'}
{'name': 'CRUD create, list and delete', 'passed': True, 'detail': 'Created trial 4, c
onfirmed the list increased by one, then deleted the temporary trial.'}
-----NODE: Supervisor-----
-----NODE: Planner-----
-----NODE: Supervisor-----
-----NODE: Reviewer-----
-----NODE: Supervisor-----
{'name': 'LangGraph terminates within timeout', 'passed': True, 'detail': 'Graph retur
ned a final state within 180 seconds.'}
{'name': 'Planner returns exactly three string tags', 'passed': True, 'detail': 'Plann
er returned 3 tags; all_string_tags=True.'}
Verification finished. Result saved to /Users/haozhewang/data260-8561/reports/hw02/ver
ification.json
```

NOTES ON DATA COLLECTION

- The main batch process was terminated by the operating system because of memory pressure during run 17.
- Ollama was restarted, and the remaining 53 runs were then completed.
- The adversarial batch became stuck during its third run because the LLM request had no timeout.
- The stuck process was terminated, and the remaining two adversarial runs were completed afterward.
- Restarting the scripts caused some `run_number` values to appear more than once.
- Results were therefore analyzed by CSV row order rather than by assuming that `run_number` was globally unique.
- A robustness limitation was identified in `model_client.py`: the `ollama.chat()` call does not define a timeout. If an LLM request hangs, the workflow can remain blocked indefinitely.
- The experiment scripts wrote each completed result to CSV immediately, so completed results were preserved when later runs failed or were terminated.

AI USE

See `reports/hw02/AI_USE.md` in the repository.

